Comprehensive Production-Grade LLMOps Project Plan

1. System Architecture & Directory Topology

The project extends the modular five-service containerized LLMOps architecture (React Frontend, FastAPI Backend, external Ollama Server, embedded FAISS Index + DuckDB, and Prometheus/Grafana/Alertmanager monitoring stack) with four production engineering upgrades:

1. **Cherry 1:** Deterministic Pydantic Output Contracts & sqlglot AST Self-Healing Execution Loop.
2. **Cherry 2:** FAISS Semantic Query Cache & Reference-Backed Intent Classification Router (Simple ML Baseline vs. Complex Text-to-SQL).
3. **Cherry 3:** Automated Pre-Merge Quality Gates (ruff, mypy, dedicated Pydantic contract script, pytest benchmark assertions) via Local Runner & GitHub Actions, backed by ephemeral Cloud GPU Smoke Tests.
4. **Cherry 4:** OpenTelemetry Distributed Span-Level Tracing across RAG, Intent Routing, SLM Inference, and DuckDB Execution.

Plaintext

```
mlops-for-small-language-models-team-2/
│
├── .github/
│   └── workflows/
│       └── quality_gates.yml          # GitHub Actions CI: Ruff, Mypy, Pydantic, Pytest Gates
│
├── docker-compose.yml                 # All 5 core services (API, frontend, monitoring)
├── Dockerfile                         # Backend multi-stage build
├── pyproject.toml                     # Unified config for Ruff, Mypy, and Pytest
├── requirements.txt                   # Production Python dependencies
├── requirements-dev.txt               # Quality gate & cloud smoke test dependencies
├── .env.example                       # Template for .env
│
├── references/                        # Cloned upstream reference architectures (gitignored)
│   ├── README.md                      # Agent instructions & git clone specification
│   ├── semantic-router/               # Pulled: aurelio-labs/semantic-router
│   ├── RouteLLM/                      # Pulled: lm-sys/RouteLLM
│   └── setfit/                        # Pulled: huggingface/setfit
```

```
│
├── scripts/
│   ├── pull_references.sh              # Clones the 3 reference repos into references/
│   ├── validate_schemas.py             # Dedicated Pydantic contract & AST verification gate
│   ├── check.sh                        # Mandatory local quality runner
(Ruff+Mypy+Pydantic+Pytest)
│   └── modal_smoke_test.py             # Ephemeral serverless GPU runner for pre-push
smoke tests
│
├── src/
│   ├── config.py                       # Central configuration module
│   ├── evaluator.py                    # Evaluator integration class
│   │
│   ├── backend/                        # FastAPI application
│   │   ├── main.py                     # App entry point, middleware stack, routers
│   │   ├── routes/
│   │   │   ├── query.py                # POST /api/v1/query - NL query execution[cite: 1]
│   │   │   ├── health.py               # GET  /api/v1/health - health checks[cite: 1]
│   │   │   ├── schema.py               # GET  /api/v1/schema - dataset schemas[cite: 1]
│   │   │   ├── compare.py              # POST /api/v1/compare - ML baseline comparison[cite: 1]
│   │   │   ├── leaderboard.py          # GET  /api/v1/leaderboard - model rankings[cite: 1]
│   │   │   └── alerts.py               # POST /api/v1/alerts/webhook - Alertmanager[cite: 1]
│   │   ├── middleware/
│   │   │   ├── auth.py                 # API key authentication[cite: 1]
│   │   │   ├── rate_limit.py           # Sliding window rate limiting[cite: 1]
│   │   │   ├── logging.py              # Request/response logging[cite: 1]
│   │   │   ├── error_handler.py        # Global error handling[cite: 1]
│   │   │   ├── metrics.py              # Custom Prometheus metrics[cite: 1]
│   │   │   ├── leaderboard_metrics.py  # Leaderboard metric tracking[cite: 1]
│   │   │   └── tracing.py              # CHERRY 4: OpenTelemetry span instrumentation
│   │   ├── schemas/                    # Pydantic request/response models[cite: 1]
│   │   │   ├── api.py                  # HTTP API boundary schemas
│   │   │   └── query.py                # CHERRY 1: SQLOutputContract + AST validation
│   │   └── services/
│   │       └── ml_baseline.py          # Random Forest & Gradient Boosting[cite: 1]
│   │
│   ├── model_customization/            # Inference-time customization[cite: 1]
│   │   ├── unified_router/
│   │   │   ├── router.py               # LLM routing + prompt construction[cite: 1]
│   │   │   ├── intent_classifier.py    # CHERRY 2: Simple vs. Complex query classifier
│   │   │   ├── semantic_cache.py       # CHERRY 2: FAISS cosine similarity query cache
│   │   │   ├── schema_rag.py           # FAISS-based schema retrieval[cite: 1]
│   │   │   └── sql_executor.py         # DuckDB execution + denormalization + self-repair[cite:
```

```
1]
│   │   └── rag/
│   │       ├── index_faiss.py            # Build FAISS vector index[cite: 1]
│   │       └── query_faiss.py            # Query FAISS index[cite: 1]
│   │
│   ├── data_preprocessing/               # Milestone 1: data pipeline[cite: 1]
│   │   └── preprocessing/
│   │       ├── main.py                   # Full preprocessing pipeline[cite: 1]
│   │       ├── parsers/                  # Dataset ingestion (Kaggle)[cite: 1]
│   │       └── processors/               # Cleaning, normalization, encoding[cite: 1]
│   │
│   ├── evaluator/                        # Milestone 5: evaluation scripts[cite: 1]
│   │   ├── train_all_models.py           # Train ML baselines (RF, GB)[cite: 1]
│   │   ├── run_evaluation.py             # Base evaluation pipeline[cite: 1]
│   │   ├── run_slm_evaluation.py         # SLM-only evaluation[cite: 1]
│   │   ├── run_slm_vs_traditional_evaluation.py # SLM vs traditional ML evaluation[cite: 1]
│   │   ├── run_slm_backbone_comparison.py    # Backbone comparison[cite: 1]
│   │   ├── save_car_queries.py           # Generate classification eval queries[cite: 1]
│   │   ├── save_housing_queries.py       # Generate regression eval queries[cite: 1]
│   │   └── visualize_*.py                # Result visualization scripts[cite: 1]
│   │
│   ├── frontend/
│   │   └── react-app/                    # React + Vite + Tailwind CSS[cite: 1]
│   │
│   └── artifacts/                        # Generated data and model artifacts[cite: 1]
│
├── monitoring/                           # Prometheus, Alertmanager, Grafana configs[cite: 1]
│
└── tests/
    ├── conftest.py                       # Pytest fixtures[cite: 1]
    ├── test_api_integration.py           # Full API integration tests[cite: 1]
    ├── test_health.py                    # Health endpoint tests[cite: 1]
    ├── test_middleware.py                # Auth, rate limiting tests[cite: 1]
    ├── test_config.py                    # Configuration validation[cite: 1]
    ├── test_schemas.py                   # Pydantic schema tests[cite: 1]
    └── quality_gates/
        ├── test_intent_routing.py        # Verifies simple vs. complex classification accuracy
        └── test_eval_thresholds.py       # CHERRY 3: Enforces ≥85% SQL & ≤10% MAPE gates
```

2. Intent Classification Architecture & Reference Repositories

To avoid fragile regex or manual heuristics when deciding whether a query is a **Simple Task** (single-record point prediction routed directly to Random Forest / Gradient Boosting baselines in src/backend/services/ml_baseline.py[cite: 1]) or a **Complex Task** (multi-condition filtering, aggregation, or cross-column analysis routed to Schema RAG + SLM Text-to-SQL + DuckDB[cite: 1]), the agent will evaluate and integrate from three established open-source implementations.

Three Concrete Reference Implementations

Method / Framework	Repository & Pull Link	Core Mechanism	Best Use Case in This Project
1. Semantic Vector Space Routing (semantic-router)	https://github.com/aurelio-labs/semantic-router.git	Encodes predefined canonical utterances for each route (simple_ml_prediction vs. complex_sql_analytics) into dense embeddings (using nomic-embed-text on Ollama[cite: 1]) and performs fast cosine similarity aggregation (RouteLayer).	**Primary Zero-Training Option:** Uses the already-required nomic-embed-text Ollama model[cite: 1] or local FastEmbed with sub-10ms latency.
2. Cost & Complexity Router (RouteLLM)	https://github.com/lm-sys/RouteLLM.git	LMSYS framework implementing Matrix Factorization (mf), BERT classification (bert), and Similarity-Weighted Elo (sw_ranking) to score query complexity and route between lightweight models and heavyweight	**Complexity Scoring Option:** Adapts the sw_ranking or mf calibrated threshold router to dispatch below-threshold complexity queries to ml_baseline.py[cite: 1] and above-threshold queries to Ollama SLMs[cite: 1].

		reasoning models.	
3. Contrastive Few-Shot Classification (SetFit)	https://github.com/huggingface/setfit.git	Fine-tunes a Sentence Transformer via contrastive learning on just 8–16 labeled queries per class (from save_car_queries.py and save_housing_queries.py[cite: 1]) followed by a lightweight classification head.	**Highest Accuracy Supervised Option:** Trains a deterministic local classifier head on your generated car and housing queries[cite: 1] with zero LLM runtime overhead.

Agent Instructions for references/README.md

Instruction to Agent: Create the references/ directory at the project root, add references/*/ (except !references/README.md) to .gitignore so third-party repositories are not committed to our Git history, and write the exact references/README.md file below. Then execute the clone commands inside references/ before writing src/model_customization/unified_router/intent_classifier.py.

Exact Content for references/README.md

Markdown

```
# Intent Classification & Query Routing Reference Implementations

This directory houses three upstream open-source repositories used as reference architectures for implementing `src/model_customization/unified_router/intent_classifier.py`. Do not commit the cloned subdirectories to version control; only this `README.md` is tracked.

## 1. Setup: Pulling Reference Repositories

Run the following commands from the project root to populate this directory:

```bash
mkdir -p references
```
```

git clone --depth 1 https://github.com/aurelio-labs/semantic-router.git references/semantic-router
git clone --depth 1 https://github.com/lm-sys/RouteLLM.git references/RouteLLM
git clone --depth 1 https://github.com/huggingface/setfit.git references/setfit

2. Reference Architecture Guide for the Agent

When implementing or refactoring src/model_customization/unified_router/intent_classifier.py, inspect the following target files in the cloned repositories:

Option A: Aurelio Labs semantic-router (Embedding Centroid / Similarity Routing)

- **Repository:** references/semantic-router
- **Key Files to Inspect:**
  - semantic_router/route.py (Route schema definition with sample utterances)
  - semantic_router/layer.py (RouteLayer similarity scoring, threshold calibration, and fallback logic)
  - semantic_router/encoders/ollama.py or semantic_router/encoders/fastembed.py
- **How to Use:** Either import semantic_router directly or adapt RouteLayer._retrieve_top_route() using our existing nomic-embed-text Ollama embeddings and FAISS store to classify queries into SIMPLE_ML_BASELINE vs. COMPLEX_SLM_SQL.

Option B: LMSYS RouteLLM (Calibrated Complexity Threshold Routing)

- **Repository:** references/RouteLLM
- **Key Files to Inspect:**
  - routellm/Controller.py (Routing orchestration between weak/simple and strong/complex targets)
  - routellm/routers/routers.py (SimilarityWeightedRouter, MatrixFactorizationRouter, BERTRouter)
  - routellm/routers/similarity_weighted/utils.py (Cosine similarity weighted scoring against calibration queries)
- **How to Use:** Reference SimilarityWeightedRouter to calculate complexity scores against our golden evaluation queries generated by save_car_queries.py and save_housing_queries.py.

Option C: Hugging Face setfit (Contrastive Few-Shot Sentence Transformer)

- **Repository:** references/setfit
- **Key Files to Inspect:**
  - src/setfit/modeling.py (SetFitModel wrapper combining SentenceTransformer body with a scikit-learn LogisticRegression head)
  - src/setfit/trainer.py (SetFitTrainer contrastive pair generation for small datasets)
- **How to Use:** If embedding distance alone yields <95% routing accuracy on unit tests, use the SetFitModel pattern (embeddings + scikit-learn LogisticRegression saved to src/artifacts/models/intent_classifier.pkl) trained on 32 labeled simple/complex car and housing queries.

3. Production Contract Requirement

Regardless of which reference implementation is used under the hood, src/model_customization/unified_router/intent_classifier.py must expose a strict Pydantic-validated interface:

Python

```
from pydantic import BaseModel, Field
from enum import Enum

class QueryRouteType(str, Enum):
    SIMPLE_ML_BASELINE = "simple_ml_baseline"
    COMPLEX_SLM_SQL = "complex_slm_sql"

class IntentClassificationResult(BaseModel):
    route: QueryRouteType
    confidence: float = Field(ge=0.0, le=1.0)
    matched_reference_method: str
```

---

3. Comprehensive Milestone-by-Milestone Expansion

The project preserves all six original LLMOps milestones[cite: 1] and expands each stage with strict typing, deterministic Pydantic contracts, and automated engineering gates.

Milestone 1: Data Preparation & Management

Original Foundation
* **Objective:** Prepare deterministic, immutable data artifacts for all downstream stages[cite: 1].
* **Datasets:**
  * **Classification:** Car Evaluation Dataset (Kaggle) predicting car acceptability[cite: 1].
  * **Regression:** California Housing Prices (Kaggle) predicting median house values[cite: 1].
* **Execution Steps:**
  1. Run `python3 -m src.data_preprocessing.preprocessing.main`[cite: 1] to generate `src/artifacts/car_eval_clean.parquet`[cite: 1], `src/artifacts/cali_housing_normalized.parquet` (min-max scaled)[cite: 1], `src/artifacts/regression_scaler.json` (min/max values for denormalization)[cite: 1], and train/eval splits (`car_eval_train.parquet`, `car_eval_eval.parquet`, `housing_train.parquet`, `housing_eval.parquet`)[cite: 1].
  2. Run `python3 src/model_customization/rag/index_faiss.py`[cite: 1] to produce `src/artifacts/faiss_store/index.faiss` and `src/artifacts/faiss_store/index.pkl`[cite: 1].
  3. Treat all generated artifacts as **read-only** in subsequent milestones[cite: 1].

Production Upgrades & Agent Tasks
1. **Pydantic Artifact & Schema Contracts:**
   * Define `RegressionScalerSchema` in `src/backend/schemas/api.py` to validate `src/artifacts/regression_scaler.json`[cite: 1] on load (ensuring `min` < `max` for every numerical feature).
   * Define `RAGChunkSchema` to validate `src/artifacts/rag_chunks.json`[cite: 1] before building the FAISS index[cite: 1].
2. **Type Safety:** Add strict Python 3.11 type annotations across `src/data_preprocessing/preprocessing/`[cite: 1] and `src/model_customization/rag/index_faiss.py`[cite: 1] so `mypy --strict` passes with zero errors.

Milestone 2: Model Selection

Original Foundation
* **Selected Backbones:** Three instruction-tuned, open-weight Small Language Models served via **Ollama** without weight fine-tuning[cite: 1]:
  * **Mistral-7B (`mistral:7b`):** Baseline model with strong instruction-following and efficiency[cite: 1].
  * **Qwen 2.5 (7B):** Comparison model selected for high SQL execution success rate[cite: 1].
  * **LLaMA 3.2:** Comparison model selected for lower latency to evaluate speed

trade-offs[cite: 1].
* **Embedding Model:** `nomic-embed-text` served via Ollama[cite: 1].
* **Selection Criteria:** Operational feasibility, open-weight licensing governance, and structured reasoning / SQL generation capability[cite: 1].

Production Upgrades & Agent Tasks
1. **Structured JSON Mode Enforcement:** Configure the Ollama client wrapper to pass `format=SQLOutputContract.model_json_schema()` on all inference requests across `mistral:7b`[cite: 1], `qwen2.5:7b`[cite: 1], and `llama3.2`[cite: 1]. This forces token sampling at the LLM runtime level to conform deterministically to our Pydantic schema.
2. **Model Configuration Validation:** Update `src/config.py`[cite: 1] using `pydantic-settings` (`BaseSettings`) to validate `OLLAMA_HOST`, `OLLAMA_USER`, and `OLLAMA_PASS`[cite: 1] at startup.

Milestone 3: Model Customization (Cherries 1 & 2)

Original Foundation
* **Constraint:** Customization is performed **at inference time only** with no modification to model weights[cite: 1].
* **Core Mechanisms:**
  1. **Schema-aware RAG retrieval (`src/model_customization/unified_router/schema_rag.py`):** FAISS retrieval and column alias resolution[cite: 1].
  2. **Structured prompt construction (`src/model_customization/unified_router/router.py`):** Injects schema context and feature values into deterministic templates and orchestrates LLM calls[cite: 1].
  3. **Execution routing (`src/model_customization/unified_router/sql_executor.py`):** Executes generated SQL against DuckDB with denormalization[cite: 1].

Production Upgrades & Agent Tasks
1. **Cherry 2A — FAISS Semantic Query Cache (`src/model_customization/unified_router/semantic_cache.py`):**
   * Before invoking Schema RAG or Ollama[cite: 1], embed the user question with `nomic-embed-text`[cite: 1] and search an in-memory FAISS query cache.
   * If cosine similarity ≥ 0.95, return the validated cached `QueryExecutionResult` in $< 15ms$.
2. **Cherry 2B — Reference-Backed Intent Classifier (`src/model_customization/unified_router/intent_classifier.py`):**
   * Implement the classifier using the patterns pulled into `references/` (`semantic-router`, `RouteLLM`, or `setfit`).
   * Route `SIMPLE_ML_BASELINE` queries (single-instance feature predictions) directly to `src/backend/services/ml_baseline.py`[cite: 1] and `COMPLEX_SLM_SQL` queries (filtering,

grouping, aggregations) to the SLM Text-to-SQL pipeline[cite: 1].
3. **Cherry 1A — Deterministic Pydantic SQL Contract (`src/backend/schemas/query.py`):**
  * Enforce `SQLOutputContract` on every LLM response:
    * Parse SQL with `sqlglot.parse_one(sql, read="duckdb")`.
    * Reject any non-`SELECT` statement (`DROP`, `DELETE`, `UPDATE`, `INSERT`, `COPY`, `ATTACH`).
    * Extract all `sqlglot.exp.Column` nodes and verify every column exists in the target dataset's schema (`car_eval_clean.parquet` or `cali_housing_normalized.parquet`[cite: 1]).
4. **Cherry 1B — Self-Healing Execution Loop (`src/model_customization/unified_router/sql_executor.py`):**
  * Wrap DuckDB execution[cite: 1] and Pydantic validation in a bounded retry loop (`max_retries = 2`).
  * If `ValidationError` (e.g., hallucinated column name) or `duckdb.Error` occurs, append the exact error traceback and valid schema columns to the prompt and request a corrected `SQLOutputContract` from Ollama[cite: 1].

Milestone 4: Application Development (Cherry 4)

Original Foundation
* **Backend (`src/backend/`):**
  * FastAPI application with 6 route modules (`query.py`, `health.py`, `schema.py`, `compare.py`, `leaderboard.py`, `alerts.py`)[cite: 1].
  * Middleware stack: CORS → Logging → Error Handling → Rate Limiting → API Key Auth[cite: 1].
  * Prometheus metrics instrumentation at `/metrics`[cite: 1].
* **Frontend (`src/frontend/react-app/`):**
  * React + Vite + Tailwind CSS served via Nginx with `/api/` proxy to backend[cite: 1].
  * Includes query interface with dataset selection, SQL preview, confidence display, and schema viewer[cite: 1].

Production Upgrades & Agent Tasks
1. **Cherry 4 — OpenTelemetry Distributed Tracing (`src/backend/middleware/tracing.py`):**
  * Instrument FastAPI and internal services with `opentelemetry-sdk` and `opentelemetry-instrumentation-fastapi`.
  * Create child spans for every stage of `/api/v1/query`[cite: 1]:
    * `span.semantic_cache_lookup`
    * `span.intent_classification`
    * `span.faiss_schema_rag`
    * `span.ollama_slm_generation` (tagged with `model.backbone`, `retry.count`)
    * `span.pydantic_sqlglot_validation`
    * `span.duckdb_execution_and_denorm`
2. **Strict API Boundary Validation (`src/backend/schemas/`):**

  * Ensure all 12 API endpoints listed in the API Reference (`/api/v1/health`, `/api/v1/query`, `/api/v1/query/validate`, `/api/v1/compare`, `/api/v1/schema`, `/api/v1/schema/{dataset}`, `/api/v1/schema/aliases`, `/api/v1/leaderboard`, `/api/v1/leaderboard/refresh`, `/api/v1/alerts/webhook`, `/api/v1/alerts`, `/metrics`)[cite: 1] use explicit `response_model` Pydantic classes with `extra="forbid"`.

Milestone 5: Evaluation & Automated Quality Gates (Cherry 3)

Original Foundation
* **Baseline Training (`src/evaluator/train_all_models.py`):** Trains Random Forest and Gradient Boosting models for both datasets and saves to `src/artifacts/models/`[cite: 1].
* **Query Generation:** `save_car_queries.py` and `save_housing_queries.py`[cite: 1].
* **Evaluation Scripts:**
  * `run_evaluation.py` (Base evaluation)[cite: 1]
  * `run_slm_evaluation.py` (SLM-only evaluation)[cite: 1]
  * `run_slm_vs_traditional_evaluation.py` (SLM vs. traditional ML comparison)[cite: 1]
  * `run_slm_backbone_comparison.py` (Mistral vs. Qwen vs. LLaMA)[cite: 1]
* **Visualization:** `visualize_evaluation_results.py` and `visualize_slm_vs_traditional_comparison.py` outputting versioned JSON artifacts in `src/artifacts/evaluation_results/`[cite: 1].

Production Upgrades & Agent Tasks
1. **Dedicated Pydantic Verification Gate (`scripts/validate_schemas.py`):**
   * Standalone script that instantiates and tests all Pydantic schemas (`SQLOutputContract`, `IntentClassificationResult`, `RegressionScalerSchema`, API request/response models) against valid payloads, adversarial SQL injections, hallucinated columns, and on-disk JSON artifacts (`src/artifacts/regression_scaler.json`, `src/artifacts/rag_chunks.json`, `src/artifacts/prompt_eval_scores.json`[cite: 1]).
2. **Mandatory Local Quality Gate Runner (`scripts/check.sh`):**
   * Must execute in strict order with `set -eo pipefail`:
     1. `ruff format src tests scripts --check`
     2. `ruff check src tests scripts`
     3. `mypy --config-file pyproject.toml src tests scripts`
     4. `python3 scripts/validate_schemas.py` *(Mandatory Pydantic script gate)*
     5. `pytest tests/ -v --cov=src --cov-report=term-missing --cov-fail-under=80`
3. **Automated Pytest Evaluation Thresholds (`tests/quality_gates/test_eval_thresholds.py`):**
   * Programmatically asserts production thresholds over evaluation artifacts (`src/artifacts/prompt_eval_scores.json` and `src/artifacts/evaluation_results/`[cite: 1]) and golden queries:
     * **SQL Execution Success Rate:** $\ge 85\%$ (far stricter than the Alertmanager runtime warning floor of 50%[cite: 1]).

    * **Schema Groundedness (Zero Hallucinated Columns):** 100%.
    * **Intent Classifier Routing Accuracy:** $\ge 92\%$ on golden test set.
    * **Denormalized Regression MAPE:** $\le 10\%$ against ground truth.
4. **GitHub Actions CI Pipeline (`.github/workflows/quality_gates.yml`):**
  * Replaces manual checks by automatically running `ruff`, `mypy`, `python3 scripts/validate_schemas.py`, unit tests, and `pytest tests/quality_gates/test_eval_thresholds.py` on every push and pull request.
5. **Ephemeral Cloud GPU Smoke Test (`scripts/modal_smoke_test.py`):**
  * Runs a serverless NVIDIA T4/L4 container on Modal that spins up Ollama, pulls the target SLM (`mistral:7b`, `qwen2.5:7b`, or `llama3.2`[cite: 1]) and `nomic-embed-text`[cite: 1], runs a smoke subset of `run_slm_evaluation.py`[cite: 1] and `test_eval_thresholds.py`, and tears down immediately so full benchmark runs only happen after smoke validation passes.

Milestone 6: Deployment & Observability

Original Foundation
* **Docker Compose Stack (`docker compose up --build`):**
  * `mlops-api`: FastAPI Backend on port `8000` (2 CPU, 2 GB RAM)[cite: 1].
  * `mlops-react-frontend`: React Frontend on port `3002` (0.5 CPU, 256 MB RAM)[cite: 1].
  * `mlops-prometheus`: Prometheus on port `9090` (1 CPU, 1 GB RAM), scraping FastAPI every 5 seconds[cite: 1].
  * `mlops-grafana`: Grafana on port `3000` (1 CPU, 512 MB RAM) with pre-provisioned Prometheus datasource[cite: 1].
  * `mlops-alertmanager`: Alertmanager on port `9093` (0.5 CPU, 256 MB RAM) routing alerts to `/api/v1/alerts/webhook`[cite: 1].
* **Volume Immutability:** All datasets and artifacts are mounted as read-only volumes[cite: 1].
* **Alert Rules (`monitoring/alerts/api_alerts.yml`):** `APIDown`, `HighLatencyP95`, `CriticalLatencyP99`, `HighErrorRate`, `CriticalErrorRate`, `HighMemoryUsage`, `LowModelAccuracy` (<30%), `LowSQLSuccessRate` (<50%)[cite: 1].
* **Security Controls:** Toggleable `API_KEY_AUTH_ENABLED` (`X-API-Key` header), sliding window `RATE_LIMIT_ENABLED` (60/min, 1000/hr returning `429 Too Many Requests`), and `CORS_ORIGINS`[cite: 1].

Production Upgrades & Agent Tasks
1. **Custom Prometheus Metrics for New Components (`src/backend/middleware/metrics.py`):**
  * Extend the existing Prometheus instrumentation[cite: 1] with:
    * `slm_semantic_cache_hits_total` and `slm_semantic_cache_misses_total`
    * `slm_intent_route_total{route="simple_ml_baseline|complex_slm_sql"}`
    * `slm_pydantic_ast_repairs_total{status="recovered|failed"}`
    * `slm_stage_duration_seconds{stage="cache|intent|rag|ollama|ast_check|duckdb"}`

2. **New Alert Rules (`monitoring/alerts/api_alerts.yml`):**
   * Add `HighPydanticValidationFailureRate` (triggers Warning if first-attempt Pydantic/SQLglot validation fails on $>15\%$ of queries over 5 minutes).

---

4. Master Agent Execution Checklist

The coding agent must execute the work in the following strict sequence:

Phase	Action Items	Verification Command
Phase 1: Reference Setup & Tooling	1. Create `references/README.md` and run `scripts/pull_references.sh` to clone `semantic-router`, `RouteLLM`, and `setfit` into `references/`. 2. Configure `pyproject.toml` (`ruff`, `mypy`, `pytest`). 3. Create `scripts/validate_schemas.py` and `scripts/check.sh`.	`bash scripts/pull_references.sh && bash scripts/check.sh`
Phase 2: Milestone 1 & 2 Hardening	1. Add Pydantic artifact validators for `regression_scaler.json`[cite: 1] and `rag_chunks.json`[cite: 1]. 2. Add `pydantic-settings` to `src/config.py`[cite: 1] and JSON schema enforcement to the Ollama client.	`python3 scripts/validate_schemas.py && mypy src/config.py src/data_preprocessing`
Phase 3: Milestone 3 Customization (Cherries 1 & 2)	1. Implement `SQLOutputContract` with `sqlglot` AST and column existence validation in `src/backend/schemas/query.py`. 2. Implement `semantic_cache.py` and `intent_classifier.py` (referencing `references/`). 3. Add the self-healing retry loop to `sql_executor.py`[cite: 1].	`pytest tests/test_schemas.py tests/quality_gates/test_intent_routing.py -v`
Phase 4: Milestone 4 & 6 Telemetry (Cherry 4)	1. Add `tracing.py` OpenTelemetry middleware to `src/backend/main.py`[cite: 1]. 2. Register cache, intent router, and AST repair counters in `src/backend/middleware/metrics.py`[cite: 1].	`pytest tests/test_api_integration.py tests/test_middleware.py -v`
Phase 5: Milestone 5 CI Gates & Cloud Smoke Test (Cherry 3)	1. Implement `tests/quality_gates/test_eval_thresholds.py` enforcing $\ge 85\%$ SQL execution rate, 100% schema groundedness, and $\le 10\%$ MAPE. 2. Create `.github/workflows/quality_gates.yml`. 3. Create `scripts/modal_smoke_test.py` for remote GPU pre-verification.	`bash scripts/check.sh && modal run scripts/modal_smoke_test.py`